

Notes on: Sensors & Actuators_from_0

1.) Voltage Considerations for Sensors

Voltage Considerations for Sensors

1. Introduction to Voltage in Sensors

- Voltage is like food or fuel for a sensor; it needs the right kind and amount to work.
- In IoT, sensors are the eyes and ears; voltage is the energy that lets them see and hear.
- Incorrect voltage can cause two main problems:

1- The sensor works poorly or gives wrong data.

2- The sensor or the microcontroller gets permanently damaged.

- Every sensor has its own specific voltage rules.
- These rules are written in a document called the **datasheet**.

2. Key Voltage Parameters

- To use a sensor correctly, you must understand these four main voltage types.
- These are always mentioned in the sensor's datasheet.
- 2.1. Operating Voltage (Vcc or Vdd)
- 2.2. Logic Level Voltage
- 2.3. Output Voltage Range
- 2.4. Input Voltage Tolerance

3. Subtopic 1: Operating Voltage (Vcc / Vdd)

- This is the main power supply voltage required by the sensor to turn on and function.
- Think of it as the sensor's required diet; e.g., it needs a 5V meal, not a 3.3V snack.
- Vcc: Usually for circuits with BJT transistors (like older TTL logic).
- Vdd: Usually for circuits with FET transistors (like modern CMOS logic).
- For us, Vcc and Vdd both mean the positive power supply pin.
- Common Operating Voltages:
- 5V: Very common for Arduino Uno and compatible sensors.
- 3.3V: Standard for modern MCUs like ESP32, ESP8266, Raspberry Pi.
- 1.8V: Used in low-power mobile devices.
- What happens if the voltage is wrong?
- Too Low Voltage:
- The sensor may not turn on at all.
- It might work unreliably; giving random or incorrect readings.
- It is like being too tired to work properly.
- Too High Voltage:
- The sensor can overheat and get permanently damaged.
- This is often called **frying** or **letting the magic smoke out**.
- This damage is irreversible.
- Example:
- A DHT11 sensor datasheet says **Supply Voltage: 3.3 - 5.5V**.
- This means you can safely power it from an Arduino's 5V pin or an ESP32's 3.3V pin.
- It is flexible.
- Rule: Always check the datasheet for the **Operating Voltage** or **Supply Voltage** range.

4. Subtopic 2: Logic Level Voltage

- This is about communication, not power.
- It is the voltage used for sending digital signals: '1' (HIGH) and '0' (LOW).
- A microcontroller and a sensor must speak the same language (same logic levels).
- Two Common Logic Level Standards:
- TTL (Transistor-Transistor Logic):
- Usually operates at 5V.
- A HIGH signal is typically above 2V.

- A LOW signal is typically below 0.8V.
- Found in older devices and boards like Arduino Uno.
- CMOS (Complementary Metal-Oxide-Semiconductor):
- Can be 5V, 3.3V, or lower.
- 3.3V is very common today.
- A HIGH signal is a large fraction of the supply voltage (e.g., >2V for a 3.3V system).
- A LOW signal is very close to 0V.
- Found in modern boards like ESP32, Raspberry Pi.
- The Problem of Mismatched Logic Levels:
- Scenario: Connecting a 5V sensor to a 3.3V microcontroller (like ESP32).
- The sensor sends a 5V signal for HIGH.
- The ESP32's input pin is only designed to handle 3.3V.
- Sending 5V to a 3.3V pin can damage the microcontroller permanently.
- The Reverse Scenario:
- Connecting a 3.3V sensor to a 5V microcontroller (like Arduino Uno).
- The sensor sends a 3.3V signal for HIGH.
- The Arduino expects a voltage above a certain threshold (e.g., 3V) to reliably read it as HIGH.
- The 3.3V signal might be high enough, but it's borderline; can lead to unreliable communication.
- The Solution: Logic Level Converter / Shifter
- A small, simple circuit board that safely converts signals between two different logic levels.
- It has a high voltage (HV) side and a low voltage (LV) side.
- You connect your 5V device to the HV side and your 3.3V device to the LV side.
- It acts as a translator between two people speaking different languages.
- Simple Alternative (One-Way): Voltage Divider
- For converting a signal from high voltage to low voltage (e.g., 5V to 3.3V).
- You can use two resistors to drop the voltage.
- This is a one-way solution only; it cannot boost a 3.3V signal up to 5V.

5. Subtopic 3: Output Voltage Range (Mainly for Analog Sensors)

- Analog sensors provide a continuous range of output voltage, not just HIGH or LOW.
- This voltage represents the physical quantity being measured (e.g., temperature, light).
- Example: An LM35 temperature sensor.
- Its output is 10mV per degree Celsius.
- At 25°C, it outputs 250mV (0.25V).
- At 100°C, it outputs 1000mV (1.0V).
- Why it is important:
- The sensor's output voltage range must match the microcontroller's ADC (Analog-to-Digital

Converter) input range.

- An ADC is what the MCU uses to read analog voltages.
- An Arduino Uno's ADC measures voltages from 0V to 5V.
- An ESP32's ADC measures voltages from 0V to 3.3V (by default).
- Mismatch Problem:
- If a sensor outputs 0-5V, but the ESP32 ADC can only read up to 3.3V...
- Any reading above 3.3V will be **clipped** and just read as the maximum value (3.3V).
- This results in incorrect data for higher-range measurements.
- Solution:
- Use a voltage divider to scale the sensor's output down to fit the ADC's range.
- Use an MCU with a compatible ADC range.
- Some MCUs allow you to change the ADC's reference voltage.

6. Subtopic 4: Power Supply Quality and Current

- The voltage supplied to a sensor should be **clean**.
- Clean Power:
- A stable DC voltage without fluctuations or electrical noise.
- Noise can come from motors, relays, or other parts of your circuit.
- This noise can corrupt the readings of sensitive analog sensors.
- Solution: Decoupling Capacitor
- A small capacitor (like 0.1μF or 100nF) placed across the Vcc and GND pins of the sensor.
- It acts like a mini-reservoir of charge, smoothing out small dips and spikes in the supply voltage.

- It's a very common and good practice in electronics.
- Current Draw (I_{cc}):
- Every sensor consumes a certain amount of electrical current to operate.
- This is measured in amperes (A), milliamperes (mA), or microamperes (μA).
- The datasheet specifies this as **Supply Current** or **Current Consumption**.
- Why it matters:
- Your power source (e.g., USB port, microcontroller's power pin) has a limited current capacity.
- An Arduino's 5V pin can typically supply a few hundred mA.
- You must ensure the total current drawn by all your sensors and components does not exceed what your power source can provide.
- If you draw too much current, the voltage will drop, and your entire system will become unstable.

7. Practical Steps & The Datasheet

• The Datasheet is the official manual for any electronic component. It is your most important resource.

- How to check requirements:
1. Find the sensor's datasheet online (e.g., [DHT11 datasheet PDF](#)).
 2. Look for the **Electrical Characteristics** or **Absolute Maximum Ratings** table.
 3. Identify **Supply Voltage** (V_{cc}) for powering the sensor.
 4. Identify **Input High Voltage** (V_{IH}) and **Input Low Voltage** (V_{IL}). This tells you the logic levels the sensor expects to receive.
 5. Identify **Output High Voltage** (V_{OH}) and **Output Low Voltage** (V_{OL}). This tells you the logic levels the sensor will send out.
 6. Compare these values with your microcontroller's datasheet.
 - Simple Checklist before connecting:
 - Does my MCU's power pin provide the correct V_{cc} for the sensor? (e.g., 5V from Arduino for a 5V sensor).
 - Do the sensor and MCU use the same logic levels? (e.g., both 3.3V).
 - If not, do I need a logic level converter? (e.g., 5V sensor with 3.3V ESP32).
 - Is the sensor's analog output range compatible with my MCU's ADC input range?

8. Summary

- Voltage is critical for sensor function and safety.
- Always check the datasheet before connecting anything.
- Match Operating Voltage (V_{cc}) between the power source and sensor.
- Match Logic Level Voltage between sensor and microcontroller, or use a level shifter.
- Ensure the power supply is clean (use decoupling capacitors) and can provide enough current.
- Mismatched voltages are a common cause of problems in IoT projects, from strange readings to destroyed components.

2.) Sensors (aim, working principle, flow, components, advantages, disadvantages) (PIR Motion Sensor, Sharp IR Distance Sensor, LDR Sensor, Gyro Sensor, Ultrasonic Distance Sensor, DHT Sensor)

SENSORS IN IOT

1- What is a Sensor?

- A sensor is a device that detects changes in its environment;
- It measures a physical property like temperature, light, or motion;
- It converts this physical measurement into an electrical signal;
- This signal can be read by a microcontroller or a computer;
- In IoT, sensors are the **senses** of a smart device; they collect data from the real world.

2- Types of Sensor Output

- Analog Sensor: Outputs a continuous range of voltage values; Example: LDR, Sharp IR sensor.
- Digital Sensor: Outputs discrete values, usually just HIGH (1) or LOW (0); Example: PIR sensor, DHT sensor.

• --

PIR MOTION SENSOR (Passive Infrared Sensor)

• --

• Aim: To detect the motion of humans or animals. It does not detect motion itself, but the moving heat signature.

- Working Principle:
 - All objects with a temperature above absolute zero emit heat energy as infrared (IR) radiation;
 - Humans and animals emit a significant amount of IR radiation;
 - The PIR sensor has two slots, each sensitive to IR;
 - When nothing is moving, both slots detect the same amount of background IR radiation;
 - When a warm body moves into its field of view, it first crosses one slot and then the other;
 - This causes a positive differential change and then a negative differential change between the two slots;
 - The sensor's electronics detect this rapid change and produce a HIGH digital output signal.

• Flow:

- 1- Sensor calibrates to the room's ambient IR level;
- 2- A warm body enters the detection area;
- 3- The Fresnel lens focuses the body's IR signature onto the pyroelectric sensor element;
- 4- The sensor detects a rapid change in IR levels;
- 5- The internal circuit processes this change;
- 6- The sensor outputs a HIGH digital signal for a fixed duration.

• Key Components:

- Fresnel Lens: The white plastic dome that increases the detection area and focuses IR rays;
- Pyroelectric Sensor: The core component that reacts to IR radiation;
- Amplifier and Circuitry: To boost the weak signal from the sensor.

• Advantages:

- Very low power consumption; ideal for battery-powered devices;
- Low cost and widely available;
- Works perfectly in complete darkness;
- High reliability and long life.

• Disadvantages:

- Cannot detect a person who is standing perfectly still;
- Can be triggered by other sources of rapid heat change (e.g., a gust of hot air);
- It cannot measure the distance to the object or how many people are there.

• --

SHARP IR DISTANCE SENSOR

• --

• Aim: To measure the distance to a solid object in front of it.

• Working Principle:

- It uses a method called triangulation;
- The sensor has an IR LED (emitter) and a Position Sensitive Detector or PSD (receiver);
- The LED sends out a continuous, narrow beam of infrared light;
- This light hits an object and reflects back at an angle;
- The PSD detects where the reflected light beam hits its surface;
- If the object is far, the reflection angle is small and the beam hits one end of the PSD;
- If the object is close, the reflection angle is large and the beam hits the other end of the PSD;

- The sensor's internal circuit converts the position on the PSD into an analog voltage.

- Flow:

- 1- IR LED emits a beam of light;
- 2- The beam reflects off an object;
- 3- A lens focuses the reflected beam onto the PSD;
- 4- The position of the light spot on the PSD is measured;
- 5- This position is converted to an analog voltage signal proportional to the distance.

- Key Components:

- IR LED Emitter: Transmits the infrared light;
- Position Sensitive Detector (PSD): The receiver that detects the reflected light's position;
- Signal Processing Circuit: Converts the PSD reading into an output voltage.

- Advantages:

- Provides a continuous analog output, giving precise distance information;
- Less affected by ambient light compared to an LDR;
- Relatively inexpensive for distance measurement.

- Disadvantages:

- The reading is affected by the object's color and reflectivity (black surfaces absorb IR light);
- It has a fixed, non-linear measurement range (e.g., 10cm to 80cm);
- Has a **dead zone** very close to the sensor where readings are unreliable.

• --

LDR SENSOR (Light Dependent Resistor)

• --

- Aim: To detect the presence and intensity of light.

- Working Principle:

- It is a special type of resistor whose resistance changes with the amount of light falling on it;
- Principle: Photoconductivity;
- In the dark, the LDR has very high resistance (Mega Ohms);
- When light falls on it, its resistance drops dramatically (a few hundred Ohms);
- It is typically used in a voltage divider circuit to get a voltage output that corresponds to light intensity.

- Flow:

- 1- Light photons strike the LDR's semiconductor surface;
- 2- The energy from photons releases electrons, increasing conductivity;
- 3- The resistance of the LDR decreases;
- 4- When used in a circuit, this resistance change causes a corresponding change in output voltage.

- Key Components:

- Photoconductive Material: Usually Cadmium Sulphide (CdS), laid out in a zigzag pattern to maximize surface area.

- Advantages:

- Extremely low cost and very simple to use;
- Available in various sizes and specifications.

- Disadvantages:

- Very slow response time; it takes time to react to changes in light;
- Not very accurate for precise light measurement (lux levels);
- Its performance is affected by temperature.

• --

GYRO SENSOR (Gyroscope)

• --

- Aim: To measure angular velocity, which is the speed of rotation around an axis.

- Working Principle:

- Modern gyros are MEMS (Micro-Electro-Mechanical Systems) based;

- They use the Coriolis Effect;

- Inside the sensor, a tiny mass (proof mass) is kept vibrating in a specific direction;

- When you rotate the sensor, the vibrating mass experiences a sideways force due to the Coriolis Effect;

- This sideways movement causes a change in capacitance between the mass and a set of fixed electrodes;

- The circuitry measures this change in capacitance and converts it into a signal representing the rate of rotation (e.g., in degrees per second).

- Flow:

- 1- An internal structure is vibrated at a known frequency;

- 2- The device is rotated externally (e.g., tilting a phone);

- 3- The Coriolis force displaces the vibrating mass;

4. The change in position is detected by capacitive sensing;

5. The output signal is proportional to the angular velocity.

- Key Components:

- MEMS Vibrating Structure: The core moving part;

- Capacitive Plates: To sense the displacement of the mass;

- Control and Sensing Circuitry.

- Advantages:

- Crucial for detecting orientation, tilt, and rotation;

- MEMS technology makes them small, cheap, and low power;

- Can measure rotation on multiple axes (e.g., X, Y, Z).

- Disadvantages:

- Suffers from **drift**, meaning small errors accumulate over time, leading to an incorrect orientation reading;

- It measures the rate of rotation, not a fixed absolute angle relative to the Earth; often combined with an accelerometer to fix this.

• --

ULTRASONIC DISTANCE SENSOR

• --

- Aim: To measure the distance to an object using sound waves.

- Working Principle:

- It operates based on the principle of SONAR (Sound Navigation and Ranging);

- The sensor has a transmitter and a receiver;

- The transmitter emits a high-frequency sound pulse (ultrasound), which is inaudible to humans;

- The sound pulse travels through the air, hits an object, and bounces back as an echo;

- The receiver detects this echo;

- The sensor measures the total time the sound wave took to travel to the object and back;

- Distance = (Speed of Sound * Time) / 2. (We divide by 2 because the time measured is for a two-way journey).

- Flow:

- 1- Microcontroller sends a short trigger pulse to the sensor;

- 2- The sensor transmits a burst of 8 ultrasonic pulses;

- 3- The sensor's ECHO pin goes HIGH;

- 4- When the reflected echo is received, the ECHO pin goes LOW;

- 5- The microcontroller measures the duration for which the ECHO pin was HIGH;
6. This time duration is used to calculate the distance.

- Key Components:
 - Ultrasonic Transmitter: Emits the sound waves;
 - Ultrasonic Receiver: Detects the echo;
 - Control Circuit: Manages the timing and communication.
- Advantages:
 - Unaffected by light, color, or transparency of the object (if it is solid);
 - Provides accurate readings for a good range (e.g., 2cm to 400cm);
 - Relatively low cost.
- Disadvantages:
 - Soft or irregular surfaces (like cloth or foam) can absorb the sound waves and give no reading;
 - Performance can be affected by changes in air temperature or humidity;
 - Has a cone-shaped detection area, so it may not detect very small objects.

• --

DHT SENSOR (Digital Humidity and Temperature Sensor)

• --

- Aim: To measure both the relative humidity and the temperature of the surrounding air.
- Working Principle:
 - For Humidity: It uses a capacitive humidity sensor. This sensor has two electrodes with a moisture-holding substrate between them. As the air's humidity changes, the substrate absorbs/releases water vapor, changing its dielectric constant and thus its capacitance. This change is measured.
 - For Temperature: It uses a thermistor. A thermistor is a resistor whose resistance changes predictably with temperature.
- Flow:
 - 1- The microcontroller sends a start signal to the DHT sensor;
 - 2- An internal chip in the DHT sensor reads the values from the humidity and temperature components;
 - 3- It converts these analog readings into a digital format;
 - 4- The chip then sends a 40-bit data packet (containing humidity, temperature, and a checksum for error checking) to the microcontroller via a single data pin.

- Key Components:
 - Capacitive Humidity Sensor;
 - Thermistor (for temperature);
 - Onboard Microcontroller: To process the data and handle communication.
- Advantages:
 - Provides two types of readings from a single, compact sensor;
 - It is a digital sensor, so it gives a direct digital output, simplifying connection to a microcontroller;
 - Low cost and very popular for hobbyist projects.
- Disadvantages:
 - Slow sampling rate; you can only request a new reading every 1-2 seconds;
 - Models like DHT11 have low accuracy and precision;
 - Not suitable for applications that require very fast and highly accurate environmental readings.

3.) Need of ADC Chip while using Analog Sensors

Topic: Need of ADC Chip while using Analog Sensors

1- Introduction: The Bridge Between Two Worlds

- In IoT, we connect physical things to the digital world.
- Analog sensors measure physical quantities; like temperature, light, pressure.
- These quantities are continuous in nature.
- For example; temperature is not just 25°C or 26°C; it can be 25.1°C, 25.11°C, etc.
- Analog sensors represent these continuous values as a continuous range of voltage.
- Example; an LDR sensor gives a low voltage in bright light and a high voltage in the dark.
- Our microcontrollers (like Arduino, ESP32) or processors (like in Raspberry Pi) are digital devices.
- They are the 'brain' of our IoT system.
- Digital brains only understand two states; 0 and 1; LOW and HIGH.
- There is a fundamental mismatch here.
- The sensor speaks an 'analog' language.
- The microcontroller speaks a 'digital' language.
- We need a translator to bridge this communication gap.
- This translator is the Analog-to-Digital Converter, or ADC.

2- Understanding Signals: Analog vs. Digital

- Analog Signal
- A continuous wave; it has infinite possible values over a given range.
- It changes smoothly over time.
- Think of it like a ramp; you can stand at any point on it.
- Analogy 1: A dimmer switch for a fan; you can set the speed to any level.
- Analogy 2: The volume knob on an old radio.
- Sensor Example: A temperature sensor's output voltage changing smoothly from 1.0V to 1.5V as the room warms up.

- Digital Signal
- A discrete signal; it has only a finite number of values.
- In computing, it is usually just two values; 0 (LOW) and 1 (HIGH).
- It changes in distinct steps.
- Think of it like a staircase; you can only stand on a specific step, not in between.
- Analogy 1: A simple on/off light switch.
- Analogy 2: A computer key; it's either pressed or not pressed.
- Sensor Example: A PIR motion sensor output is HIGH (e.g., 3.3V) when motion is detected, and LOW (0V) when there is no motion.

3- The Core Problem: Why Conversion is Necessary

- A microcontroller is a digital computer.
- Its CPU can only process binary numbers (sequences of 0s and 1s).
- It cannot directly understand a voltage value like 2.75V from a sensor.
- To the microcontroller, 2.75V is not 0 and it is not 1; it is a meaningless value.
- For the microcontroller to process sensor data, that data must be in a digital format.
- It needs a number it can store in memory and use in calculations.
- Without conversion, the rich, detailed information from the analog sensor is lost.
- The microcontroller would not know the difference between 'a little bright' and 'very bright' from an LDR.

4- The Solution: The Analog-to-Digital Converter (ADC)

- What is an ADC?
- An ADC is an electronic circuit or a chip.
- Its main job is to convert an analog voltage signal into a digital number.
- It acts as an interpreter between the analog sensor and the digital microcontroller.
- How does it work (in simple terms)?

- Step 1: Sampling - The ADC measures (samples) the analog voltage at a specific point in time. It's like taking a quick photo of the voltage level.
- Step 2: Quantization - The ADC then maps this measured voltage to the nearest digital value it can create. It's like rounding off a number.
- Step 3: Encoding - Finally, it converts this digital value into a binary number that the microcontroller can read and process.

5- Key Parameters of an ADC

- These parameters decide how good the ADC is.
- A. Resolution
 - This is the most important characteristic.
 - It defines the number of discrete steps the ADC can use to represent the analog signal.
 - It is measured in 'bits'.
 - A higher bit count means higher resolution; which means more steps and a more accurate measurement.
 - The number of steps = $2^{\text{(Resolution in bits)}}$.
 - Example 1: An 8-bit ADC.
 - Number of steps = $2^8 = 256$ steps.
 - It can represent the analog voltage with digital values from 0 to 255.
 - Example 2: A 10-bit ADC (very common in Arduino Uno).
 - Number of steps = $2^{10} = 1024$ steps.
 - It can represent the analog voltage with digital values from 0 to 1023.
 - Example 3: A 12-bit ADC.
 - Number of steps = $2^{12} = 4096$ steps.
 - It can represent the analog voltage with digital values from 0 to 4095.
 - Analogy:
 - A low-resolution ADC is like a ruler with only centimeter marks. You can measure 5 cm or 6 cm, but not 5.5 cm accurately.
 - A high-resolution ADC is like a ruler with millimeter marks. It is much more precise.
- B. Reference Voltage (Vref)
 - This is the maximum voltage the ADC can measure.
 - The ADC scales the entire range of digital values (e.g., 0-1023) across this voltage range (e.g., 0V to Vref).
 - Vref sets the 'top end' of the measurement scale.
 - For many microcontrollers like Arduino, the Vref is typically 5V or 3.3V.
- How it all works together: A Practical Example
 - Let's use an Arduino Uno.
 - It has a 10-bit ADC. (So, 1024 steps from 0 to 1023).
 - Its Vref is 5V.
 - An analog sensor is connected and it outputs 2.5V.
 - The ADC will convert this.
 - Calculation: Digital Value = (Input Voltage / Vref) * (Number of Steps)
 - Digital Value = $(2.5V / 5.0V) * 1024$
 - Digital Value = $0.5 * 1024$
 - Digital Value = 512
 - The microcontroller reads the number 512.
 - In your code, you can use this number to determine the physical property.
 - If the sensor output was 0V, the digital value would be 0.
 - If the sensor output was 5V, the digital value would be 1023.
- C. Sampling Rate

- This tells you how many times per second the ADC can perform a conversion.
- It is measured in Samples Per Second (SPS).
- For slow-changing signals like room temperature, a low sampling rate is fine.
- For fast-changing signals like audio or vibration, a high sampling rate is essential to capture the signal accurately.

6- Practical Implementation: Where is the ADC located?

- There are two main ways to use an ADC in your IoT project.
 - A. Built-in (Internal) ADC
 - Most modern microcontrollers have an ADC built directly into the chip.
 - For example, Arduino Uno, Nano, and ESP32 have built-in ADCs.
 - The pins for these are usually marked 'A0', 'A1', 'A2', etc. (for Analog).
 - This is the most common and convenient way for students and hobbyists.
 - You just connect the analog sensor's output pin directly to one of these analog pins.
 - The resolution is usually fixed (e.g., 10-bit for Arduino, 12-bit for ESP32).
 - B. External ADC Chip
 - Sometimes, the built-in ADC is not good enough for a specific application.
 - You might need an external, dedicated ADC chip.
 - Reasons to use an external ADC:
 1. Higher Resolution: For scientific instruments or high-precision tasks, you might need a 16-bit, 20-bit, or even 24-bit ADC.
 2. More Channels: If your project has many analog sensors and your microcontroller doesn't have enough analog pins.
 3. Better Performance: External ADCs can have lower noise and higher stability.
 4. No Built-in ADC: Some boards, like the Raspberry Pi, do not have any built-in ADC at all. You MUST use an external ADC to connect any analog sensor to it.
 - Popular external ADC chips: MCP3008, ADS1115.
 - These chips communicate with the main microcontroller using standard digital protocols like SPI or I2C.

7- Conclusion

- An ADC is an essential component in almost every IoT system that interacts with the real world.
- It translates the continuous language of analog sensors into the discrete, binary language of microcontrollers.
- Without an ADC, the digital brain of your IoT device would be deaf to the signals from analog sensors.
- The choice between a built-in ADC and an external ADC depends on your project's requirements for precision, speed, and the number of sensors.
- Understanding the role of the ADC is fundamental to successfully reading data from a wide variety of sensors and making your IoT project truly smart.

4.) Actuators (aim, working principle, flow, components, advantages, disadvantages)(Servo Motor, Solenoid, Stepper Motor)

Actuators in IoT

- Introduction to Actuators
- An actuator is a component of a system that is responsible for moving or controlling a mechanism or system.
- It is the **muscle** or **action** part of an IoT system.

- While sensors collect data from the environment (senses); actuators act on the environment.
- Actuators take an electrical signal from a controller (like a microcontroller) and convert it into physical energy; usually motion.
- Example Flow: A Temperature Sensor (DHT) senses high temperature; sends data to a microcontroller; the microcontroller processes this data and sends a signal to an actuator (a fan motor) to turn ON.

- Aim of an Actuator
- To convert energy (usually electrical) into physical motion or action.
- The motion can be linear (in a straight line); rotary (in a circle); or oscillatory (back and forth).
- To interact with and change the physical world based on digital commands.
- General Working Principle
- An actuator receives a control signal; typically a low-voltage electrical signal from a microcontroller.
- This signal triggers a mechanism inside the actuator.
- The mechanism converts stored or input energy (like electrical current) into mechanical force or movement.
- The output is a physical action; like rotating a shaft; pushing a lever; or opening a valve.
- General Components
- Energy Source: Usually electrical power from a power supply.
- Control Signal Input: The terminal that receives commands from the microcontroller.
- Conversion Mechanism: The core part that changes energy to motion (e.g., motor coil, electromagnet).
- Mechanical Output: The physical part that moves (e.g., motor shaft, plunger).

• --
1. Servo Motor
• --

- Aim
- To provide precise control over angular position.
- To rotate a shaft to a specific angle and hold it there.
- Not for continuous rotation like a normal DC motor; but for position control.
- Commonly used for angles between 0 and 180 degrees.
- Working Principle
- It is a closed-loop control system; meaning it has internal feedback.
- It consists of a DC motor; a gearbox; a position sensor (potentiometer); and a control circuit.
- The control circuit receives a PWM (Pulse Width Modulation) signal from a microcontroller.
- The width of the pulse tells the servo what angle to move to.
- - 1ms pulse: 0 degrees (example).
- - 1.5ms pulse: 90 degrees (example).
- - 2ms pulse: 180 degrees (example).
- The internal potentiometer checks the current position of the shaft.
- The control circuit compares the target position (from PWM signal) with the current position (from potentiometer).
- If they are different; the circuit drives the DC motor until the positions match.
- Once the position is reached; the motor stops.
- If an external force tries to change the position; the circuit will power the motor to resist the change and hold the position.
- Control Flow
- Microcontroller (e.g., Arduino) generates a specific PWM signal.
- The signal is sent to the servo's signal pin.
- Servo's internal circuit decodes the pulse width to determine the target angle.
- The circuit drives the internal DC motor.
- The gearbox reduces the motor's high speed to a slower; higher-torque output.
- The potentiometer continuously feeds back the current shaft angle to the circuit.

- The circuit stops the motor when the current angle equals the target angle.

- Components

- DC Motor: The primary source of motion.
- Gear Reduction Unit: A set of gears to reduce speed and increase torque (turning force).
- Position Sensor: A potentiometer connected to the final output shaft to measure the angle of rotation.

rotation.

- Control Circuit: An electronic board that reads the PWM signal and controls the motor.
- Output Shaft: The part that rotates and connects to other mechanical parts.

- Advantages

- Precise angular positioning is possible.
- Simple to control with a single PWM signal pin from a microcontroller.
- High torque at low speeds due to the gearbox.
- It is a self-contained; closed-loop system; no external feedback sensor is needed.

- Disadvantages

- Limited rotation range; typically 0-180 degrees.
- Can produce a buzzing noise as it constantly corrects its position.
- Consumes power even when idle to hold its position.
- Gearbox can be a point of failure; especially plastic gears under high load.

- --

2. Solenoid

- --

- Aim

- To produce a short; linear (push or pull) motion.
- To act as an electronic switch for mechanical systems; like opening a latch or valve.

- Working Principle

- Based on the principle of electromagnetism.
- A solenoid is a coil of wire wrapped around a hollow cylinder.
- Inside the cylinder is a movable iron or steel core called a plunger or armature.
- When an electric current flows through the coil; it creates a strong magnetic field.
- This magnetic field pulls the plunger into the center of the coil.
- This movement can be used to push or pull something.
- When the current is turned off; the magnetic field disappears.
- A spring usually returns the plunger to its original position.
- It is a binary (ON/OFF) actuator.

- Control Flow

- Microcontroller sends a HIGH signal to a transistor or relay (since solenoids need more current).
- The transistor/relay allows a larger current to flow through the solenoid's coil.
- The coil energizes; creating a magnetic field.
- The plunger is pulled in (or pushed out; depending on the type).
- The linear motion performs an action (e.g.; unlatching a lock).
- Microcontroller sends a LOW signal; stopping the current.
- The magnetic field collapses; and the return spring pushes the plunger back.

- Components

- Coil: A copper wire wound into a tight helix.
- Frame or Housing: The structure that holds the coil.
- Plunger (Armature): The movable ferromagnetic core that slides in and out.
- Spring: Optional but common; used to return the plunger to its initial state.

- Advantages

- Very fast response time; motion is almost instantaneous.
- Simple design and operation; very reliable.

- Low cost and widely available.
- High force in a small package.

- Disadvantages
- Very short and limited range of motion (stroke length).
- It is a binary device; only two positions (ON/OFF or in/out).
- Can overheat if kept energized for a very long time.
- Can be noisy in operation (produces a **click** or **thud** sound).

• --

3. Stepper Motor

• --

- Aim
- To achieve precise rotational movement in a series of discrete steps.
- To control rotation angle and speed with high accuracy without a feedback sensor.

• Working Principle

- It is an open-loop control system.
- The motor's internal structure has a rotor (the rotating part) with magnets and a stator (the stationary part) with multiple electromagnetic coils.
- These coils are organized in groups called **phases**.
- The motor moves by energizing these coils in a specific sequence.
- When one phase is energized; it creates a magnetic field that attracts the rotor's teeth; causing it to align.
- To rotate the motor; the controller de-energizes the first phase and energizes the next one in the sequence.
- The rotor then rotates by a small; fixed angle (one **step**) to align with the new magnetic field.
- By continuously sending a sequence of pulses to energize the coils in order; the motor rotates step-by-step.
- The speed of rotation depends on the frequency of these pulses.
- The direction of rotation depends on the order of the sequence.

• Control Flow

- Microcontroller determines the required number of steps and direction.
- It generates a sequence of digital pulses.
- These pulses are sent to a dedicated stepper motor driver circuit (e.g.; ULN2003, A4988).
- A driver is needed because the motor coils require more current than a microcontroller can supply.
- The driver receives the simple step/direction signals and energizes the correct motor coils in the correct sequence.
- For each pulse from the microcontroller; the driver advances the sequence; causing the motor to move one step.

• Components

- Stator: The outer casing with sets of electromagnetic coils.
- Rotor: The central shaft with permanent magnets or teeth made of a magnetic material.
- Bearings: To allow the rotor to spin smoothly.
- Requires an external component: A Stepper Motor Driver.

• Advantages

- Excellent precision in positioning; the angle of rotation is directly related to the number of input pulses.
- High torque at low speeds (standstill); known as holding torque. This allows it to hold a position firmly against a force.
- It is an open-loop system; which makes the control electronics simpler (no feedback sensor needed).
- Reliable and has a long life due to the absence of brushes (in most common types).

• Disadvantages

- Control is more complex than a simple DC motor; requires a driver circuit and pulse sequencing.
- Can miss steps if the load is too high or the pulse rate is too fast; causing position errors (a key problem of open-loop systems).
- Consumes considerable power even when not moving (to maintain holding torque).
- Can vibrate or be noisy; especially at low speeds.
- Lower torque at higher speeds.

5.) Need of Relay while using Actuators

Need of Relay while using Actuators

1- Introduction: The Core Problem

- We have a brain for our IoT system; a microcontroller like Arduino or Raspberry Pi.
- This brain is smart; but it is physically weak.
- We have muscles for our system; actuators like motors, solenoids, or pumps.
- These muscles are strong; but they need a strong signal to work.
- The microcontroller's signal is too weak to directly power most actuators.
- This is the main problem; a mismatch of power.

2- Understanding the Mismatch: Power Levels

- Microcontroller Output:
 - Works on low DC voltage; typically 3.3V or 5V.
 - Provides very low current; typically 20-40 milliamperes (mA).
 - This is enough for small things; like an LED.
- Actuator Requirement:
 - Many actuators need high voltage; like 12V, 24V DC, or even 220V AC (for home appliances).
 - They require high current; from hundreds of mA to several amperes (A).
 - Example: A small DC motor might need 12V and 500mA.
 - Example: A water pump might need 220V AC and 2A.
- The Consequence of Direct Connection:
 - If you connect a high-power actuator directly to a microcontroller pin...
 - The actuator will try to draw too much current.
 - This will permanently damage or destroy the microcontroller.
 - Think of it like trying to start a truck engine with a tiny watch battery.

3- The Solution: The Relay

- What is a Relay?
 - A relay is an electrically operated switch.
 - It's a switch you can turn ON or OFF with an electrical signal; not by hand.
 - It uses a small amount of power to control a large amount of power.
 - It acts as a bridge; connecting the low-power world of the microcontroller to the high-power world of the actuator.
- A Simple Analogy:
 - Imagine you are a person (the microcontroller); very smart but not very strong.
 - You want to lift a very heavy box (the actuator).
 - You can't lift it yourself.
 - So you hire a very strong person (the relay).
 - You just have to whisper **lift** (low-power signal) to the strong person.
 - The strong person then uses their strength to lift the heavy box (high-power action).
 - You are safely separated from the heavy work.

4- How an Electromechanical Relay Works

- A relay has two separate circuits:
 - 1. Control Circuit (Low-Power Side):
 - This is where the microcontroller connects.
 - It has a coil of wire.
 - 2. Load Circuit (High-Power Side):
 - This is where the actuator and its power supply connect.
 - It has a physical switch with contacts.
- Key Components:
 - Coil (Electromagnet): A copper wire wrapped around an iron core.
 - Armature: A movable metal lever.
 - Spring: Pulls the armature back to its default position.
 - Contacts: The actual switch points.
 - - COM: Common terminal.
 - - NO: Normally Open terminal.
 - - NC: Normally Closed terminal.
- Step-by-Step Working:
 - Step 1: Microcontroller sends a small current (e.g., 5V signal) to the coil.
 - Step 2: The coil becomes an electromagnet; it creates a magnetic field.
 - Step 3: This magnetic field attracts the metal armature.
 - Step 4: The armature pivots, like a see-saw.
 - Step 5: This movement flips the switch on the high-power side.
 - Step 6: It connects the COM terminal to the NO (Normally Open) terminal.
 - Step 7: The high-power circuit is now complete.
 - Step 8: The actuator, connected to this circuit, turns ON.
 - Step 9: When the microcontroller stops the current to the coil...
 - Step 10: The magnetic field disappears.
 - Step 11: The spring pulls the armature back to its original position.
 - Step 12: The switch opens; the high-power circuit is broken.
 - Step 13: The actuator turns OFF.

5- Summary of Reasons: Why a Relay is Essential

- 1. Power Switching / Current Amplification:
 - The main reason.
 - A small current from the microcontroller (e.g., 20mA) controls a large current for the actuator (e.g., 5A).
 - It lets a weak signal control a powerful device.
- 2. Electrical Isolation:
 - This is a very important safety feature.
 - The control circuit (microcontroller side) and the load circuit (actuator side) are physically separate.
 - There is no electrical connection between them; only a magnetic one.
 - This protects the sensitive microcontroller from the **dirty** and high power of the actuator's circuit.
 - If something goes wrong on the high-power side, the microcontroller is safe.
- 3. Voltage Switching:
 - Allows a low-voltage microcontroller (e.g., 5V DC) to switch a high-voltage actuator.
 - This high voltage can be 12V DC, 24V DC, or even 230V AC from your home's wall socket.
 - You can control home appliances like lights, fans, and pumps with an Arduino.
- 4. AC and DC Control:
 - Microcontrollers only work with DC (Direct Current).
 - Many powerful actuators and appliances work with AC (Alternating Current).

- A relay can switch either AC or DC loads.
- This makes it very versatile for real-world projects.
- 5. Protection from Back EMF (Inductive Kickback):
- Actuators like motors and solenoids are **inductive loads**.
- When you suddenly turn them off, the collapsing magnetic field in their coils creates a large, reverse voltage spike.
- This spike is called Back EMF.
- This voltage spike can be very high and will flow back into your circuit.
- If connected directly, it would instantly destroy your microcontroller.
- The relay provides isolation, protecting the microcontroller from this kickback.
- Note: A flyback diode is often used with the relay coil itself for protection.

6- Practical Scenario: Controlling a Table Fan with an Arduino

- The Goal: Turn a 220V AC table fan ON and OFF using an Arduino.
- The Actuator: The 220V AC table fan.
- The Controller: The Arduino board.
- The Setup:
- We use a **Relay Module** for this. It's a small board with the relay and all necessary components.
- The Relay Module has two sides.
- Low-Power Side Connections:
- - VCC pin to Arduino 5V pin.
- - GND pin to Arduino GND pin.
- - IN (Input) pin to an Arduino digital pin (e.g., Pin 7).
- High-Power Side Connections (BE VERY CAREFUL WITH 220V AC):
- - The fan has a power cord with two wires (Live and Neutral).
- - Cut the Live wire of the fan's power cord.
- - Connect one end of the cut Live wire to the COM terminal of the relay.
- - Connect the other end of the cut Live wire to the NO (Normally Open) terminal.
- - The Neutral wire of the fan remains uncut and goes directly to the wall socket.
- The Code (Logic):
- To turn the fan ON:
- - `digitalWrite(7, HIGH);`
- - This sends a 5V signal to the relay module.
- - The relay 'clicks' ON.
- - The COM and NO terminals get connected.
- - The Live wire circuit for the fan is completed.
- - The fan starts running.
- To turn the fan OFF:
- - `digitalWrite(7, LOW);`
- - This sends a 0V signal to the relay module.
- - The relay 'clicks' OFF.
- - The COM and NO terminals are disconnected.
- - The fan's circuit is broken.
- - The fan stops.

7- Conclusion

- A relay is a mandatory component when a microcontroller needs to control a high-power actuator.
- It's not just a switch; it's a translator and a bodyguard for your electronics.
- It translates a low-power command into a high-power action.
- It protects your low-voltage microcontroller from the dangerous high-voltage world.
- Without relays, most practical and powerful IoT applications would be impossible.